```python
# Lab 5 - Appendix: Additional Exercises
# Name: Bacatan | ID: TUPM-26-3609 | Favorite artist: Hev Abi
import random
import time

# ---------- Student-specific inputs ----------
LAST_NAME = "Bacatan"
STUDENT_ID = "TUPM-26-3609"
FAVORITE_ARTIST = "Hev Abi"

digits = [int(c) for c in STUDENT_ID if c.isdigit()]
letters = [c for c in FAVORITE_ARTIST.upper() if c.isalpha()]  # ignore the space

SEED_DIGIT = digits[-1]                        # last digit of ID
ID_SUM = sum(digits)                           # sum of ID digits
NAME_LENGTH = len(LAST_NAME)
ARTIST_LENGTH = len(letters)
ARTIST_SUM = sum(ord(c) - 64 for c in letters)  # A=1 ... Z=26

print("SEED_DIGIT   =", SEED_DIGIT)
print("ID_SUM       =", ID_SUM)
print("NAME_LENGTH  =", NAME_LENGTH)
print("ARTIST_LENGTH=", ARTIST_LENGTH)
print("ARTIST_SUM   =", ARTIST_SUM)


# ====================================================
# 1. EMERGENCY DISPATCH RECORD SEARCH (rotated list)
# ====================================================
def linear_search(data, target):
    comparisons = 0
    for i in range(len(data)):
        comparisons += 1
        if data[i] == target:
            return i, comparisons
    return -1, comparisons


def rotated_binary_search(data, target):
    low, high = 0, len(data) - 1
```

```python
def rotated_binary_search(data, target):
    low, high = 0, len(data) - 1
    comparisons = 0
    while low <= high:
        mid = (low + high) // 2
        comparisons += 1
        if data[mid] == target:
            return mid, comparisons
        if data[low] <= data[mid]:              # left half is sorted
            if data[low] <= target < data[mid]:
                high = mid - 1
            else:
                low = mid + 1
        else:                                   # right half is sorted
            if data[mid] < target <= data[high]:
                low = mid + 1
            else:
                high = mid - 1
    return -1, comparisons


print("\n=== 1. Emergency Dispatch Record Search ===")
n = 10
original = [ID_SUM + NAME_LENGTH * i for i in range(n)]   # unique, ascending
k = SEED_DIGIT % n                                          # rotation point
rotated = original[k:] + original[:k]
target = original[(ID_SUM + ARTIST_SUM) % n]                # target incident

pos_b, comp_b = rotated_binary_search(rotated, target)
pos_l, comp_l = linear_search(rotated, target)

print("Original dataset :", original)
print("Rotation point   :", k)
print("Rotated dataset  :", rotated)
print("Target incident  :", target)
if pos_b == -1:
    print("Result           : Not found")
else:
    print("Position found   : index", pos_b)
print("Binary comparisons:", comp_b)
print("Linear comparisons:", comp_l)
```

```python
# ====================================================
# 2. UNIVERSITY ENROLLMENT RECORDS (merge two sorted lists)
# ====================================================
def merge(a, b):
    i = j = comparisons = 0
    result = []
    while i < len(a) and j < len(b):
        comparisons += 1
        if a[i] <= b[j]:
            result.append(a[i])
            i += 1
        else:
            result.append(b[j])
            j += 1
    result += a[i:]
    result += b[j:]
    return result, comparisons


print("\n=== 2. University Enrollment Records ===")
size = 8
dept_a = [ID_SUM + SEED_DIGIT * i for i in range(size)]
dept_b = [NAME_LENGTH + ARTIST_LENGTH * i for i in range(size)]
merged, comps = merge(dept_a, dept_b)

print("Department A     :", dept_a)
print("Department B     :", dept_b)
print("Consolidated     :", merged)
print("Records before   :", len(dept_a), "+", len(dept_b), "=", len(dept_a) + len(dept_b))
print("Records after    :", len(merged))
print("Comparison count :", comps)
print("Ordered correctly:", merged == sorted(dept_a + dept_b))
print("Time complexity  : O(n + m)")
print("Space complexity : O(n + m)")


# ====================================================
# 3. E-COMMERCE ORDER PROCESSING (3 sorts x 3 conditions)
# ====================================================
def bubble_sort(arr):
    arr = arr[:]
```

```python
# ==================================================
# 3. E-COMMERCE ORDER PROCESSING (3 sorts x 3 conditions)
# ==================================================
def bubble_sort(arr):
    arr = arr[:]
    comps = moves = 0
    for i in range(len(arr) - 1):
        for j in range(len(arr) - 1 - i):
            comps += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                moves += 1
    return arr, comps, moves


def selection_sort(arr):
    arr = arr[:]
    comps = moves = 0
    for i in range(len(arr) - 1):
        small = i
        for j in range(i + 1, len(arr)):
            comps += 1
            if arr[j] < arr[small]:
                small = j
        if small != i:
            arr[i], arr[small] = arr[small], arr[i]
            moves += 1
    return arr, comps, moves


def insertion_sort(arr):
    arr = arr[:]
    comps = moves = 0
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0:
            comps += 1
            if arr[j] > key:
                arr[j + 1] = arr[j]
                moves += 1
                j -= 1
```

```
            else:
                break
        arr[j + 1] = key
    return arr, comps, moves


print("\n=== 3. E-Commerce Order Processing ===")
random.seed(ID_SUM + ARTIST_SUM)
orders = random.sample(range(1000, 1000 + ID_SUM * 10), 12)   # unique order numbers
conditions = {
    "Sorted": sorted(orders),
    "Reverse": sorted(orders, reverse=True),
    "Random": orders,
}
algorithms = {
    "Bubble": bubble_sort,
    "Selection": selection_sort,
    "Insertion": insertion_sort,
}

print("Generated orders:", orders)
print(f"{'Condition':<9} {'Algorithm':<10} {'Comps':>6} {'Moves':>6} {'Time(s)':>10}  Sorted?")
for cname, data in conditions.items():
    for aname, func in algorithms.items():
        start = time.perf_counter()
        result, comps, moves = func(data)
        elapsed = time.perf_counter() - start
        ok = result == sorted(data)
        print(f"{cname:<9} {aname:<10} {comps:>6} {moves:>6} {elapsed:>10.6f}  {ok}")

print("\nBig O / case analysis:")
print("Bubble    : best O(n) if early-exit (this version O(n^2)), avg/worst O(n^2)")
print("Selection : best/avg/worst O(n^2), fewest moves")
print("Insertion : best O(n) (sorted), avg/worst O(n^2) (random/reverse)")
print("Space     : O(1) for all three")
```

✓ 0.0s

```
SEED_DIGIT   = 9
ID_SUM       = 26
NAME_LENGTH  = 7
ARTIST_LENGTH= 6
ARTIST_SUM   = 47

=== 1. Emergency Dispatch Record Search ===
Original dataset : [26, 33, 40, 47, 54, 61, 68, 75, 82, 89]
Rotation point   : 9
Rotated dataset  : [89, 26, 33, 40, 47, 54, 61, 68, 75, 82]
Target incident  : 47
Position found   : index 4
Binary comparisons: 1
Linear comparisons: 5
Time complexity  : O(log n) (binary) vs O(n) (linear)
Space complexity : O(1)

=== 2. University Enrollment Records ===
Department A     : [26, 35, 44, 53, 62, 71, 80, 89]
Department B     : [7, 13, 19, 25, 31, 37, 43, 49]
Consolidated     : [7, 13, 19, 25, 26, 31, 35, 37, 43, 44, 49, 53, 62, 71, 80, 89]
Records before   : 8 + 8 = 16
Records after    : 16
Comparison count : 11
Ordered correctly: True
...
Bubble    : best O(n) if early-exit (this version O(n^2)), avg/worst O(n^2)
Selection : best/avg/worst O(n^2), fewest moves
Insertion : best O(n) (sorted), avg/worst O(n^2) (random/reverse)
Space     : O(1) for all three
```